Lightweight Prefetch-Resilient LLC Replacement Policy for Multicore Systems

Name : Sharma Chirag Krishnakumar
Roll No : S20240010223
Guide : Dr. Bheemappa Halavar (BTP)

The Problem

Cache Pollution from Aggressive Prefetching

- Prefetchers speculatively fetch data based on patterns (stride, streams)
- Wrong predictions = wasted cache space → useful demand data evicted
- Multicore amplifies: 4 cores × prefetchers = 4× pollution
- Example: mcf pointer-chasing defeats stride prefetchers
- Result: Increased LLC misses, wasted bandwidth, reduced performance

Literautre & Research Gaps

- Drishti (MICRO 2025):
- Per-core reuse prediction (GPHT) for sliced LLCs
- Dynamic policy selection framework
- Gap: Doesn't handle prefetch vs. demand distinction

PACMan (IEEE CAL 2023):

- Tracks prefetch accuracy to modulate insertion/promotion
- Works well but adds hardware overhead
- Gap: No lightweight zero-cost alternative explored

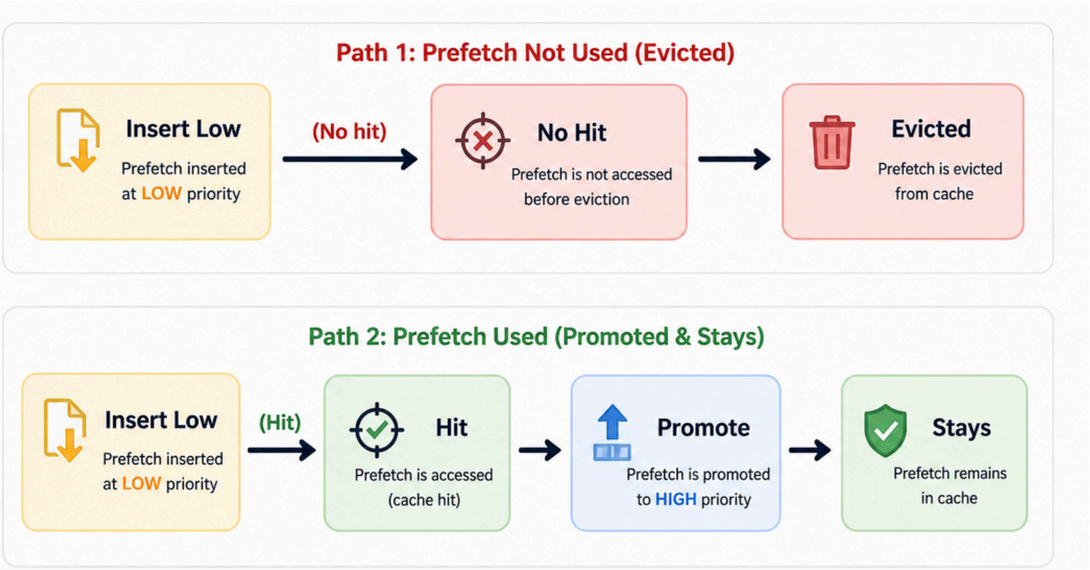
Our Question: Can we achieve prefetch-awareness with zero hardware overhead through simple insertion priority?

Propose Approach

Low-Priority Insertion + Promotion on Hit

Three-Step Mechanism:

1. Prefetch blocks inserted NEAR LRU → evicted first if unused
2. Demand blocks inserted at MRU → normal LRU behavior
3. On cache hit: promote to MRU → useful prefetches stay



Key Properties:

- Zero new hardware (uses existing access_type flag)
- Self-adaptive (rewards useful prefetches automatically)
- Prefetcher-agnostic (works with any prefetcher)
- Simple (~100 lines of code)

Implementation

ChampSim Replacement Module: lru_prefetch_aware

Built Configurations:

- champsim_4core_baseline (ip_stride + standard LRU)
- champsim_4core_prefetch_aware (ip_stride + your policy)
- champsim_4core_next_line (next_line prefetcher variant)

```
// On prefetch fill: Insert at LOW priority
if (type == PREFETCH) {
  last_used_cycles[index] = min_in_set + 1;
}

// On cache hit: Promote if it was a prefetch
if (hit && is_prefetched[index]) {
  prefetch_promoted++;  // Track usefulness
    last_used_cycles[index] = cycle++;   // Move to
MRU
}
```

Results — Single Core

gcc (Stride-Friendly) vs. mcf (Pointer-Chasing)

GCC – Mixed Workload (High Prefetch Accuracy):

Metric	Baseline	Prefetch-Aware	Change
IPC	1.967	1.976	+0.46% ✓
Demand Misses	1,534	1,444	-90 fewer
Prefetch Fills	—	163	—
Usefulness %	—	35%	57 promoted

MCF – Pointer-Chasing (Low Prefetch Accuracy):

Metric	Baseline	Prefetch-Aware	Change
IPC	0.0743	0.0743	+0.07% (flat)
Demand Misses	382,694	382,593	-101 fewer
Prefetch Fills	—	478	—
Usefulness %	—	0.2%	1 promoted

Key Insight: Policy adapts automatically—high usefulness (gcc) improves IPC; low usefulness (mcf) protects demand data

Result - Multi Core

4-Core Heterogeneous: gcc + mcf + perlbench + omnetpp

Per-Core IPC (Shared 8 MB LLC):

Core	Trace	Type	Baseline	Prefetch-Aware
0	gcc	Mixed	1.970	1.971
1	mcf	Memory-Bound	0.0847	0.0847
2	perlbench	Compute	2.027	2.027
3	omnetpp	Mixed	0.207	0.207

Shared LLC Statistics:

- Total prefetch fills: 6,475
- Prefetch blocks promoted (useful): 2,088
- Overall usefulness: 32.2% ← Stable & consistent with single-core gcc (35%)

Critical Finding: Policy maintains self-adaptive behavior even under 4-core contention; no per-core tuning needed

Key Insight

Self-Adaptive Behavior Without Any Explicit Predictor

Same Policy, Different Behavior:

Stride-Friendly (gcc) – 35% Useful Prefetches:

- Prefetch arrives → Insert LOW
- 35% receive cache hit → Promoted to MRU
- Result: Useful prefetches accumulate → IPC +0.46%

Pointer-Chasing (mcf) – 0.2% Useful Prefetches:

- Prefetch arrives → Insert LOW
- 0.2% receive cache hit → Promoted to MRU
- Result: Useless prefetches evicted → Demand protected

Why This Is Powerful:

- No explicit prediction hardware (unlike Drishti/PACMan)
- Cache hits/misses themselves provide the feedback signal
- Works across any prefetcher type (stride, next-line, SPP, AMPM)
- Automatic phase adaptation (no retuning needed)

Summary

Objective 1: Zero-Overhead Design

- Implemented lru_prefetch_aware replacement module
- Cost: ~100 lines of code, zero new hardware structures
- Uses only existing access_type flag

Objective 2: Diverse Evaluation

- Single-core: perlbench (compute), gcc (mixed), mcf (memory)
- Multicore: 4-core heterogeneous (gcc + mcf + perlbench + omnetpp)
- Metrics: IPC, LLC MPKI, prefetch usefulness (%)

Objective 3: Characterization

- Effective for stride-friendly workloads (gcc: 35% usefulness, +0.46% IPC)
- Self-protective on hostile workloads (mcf: 0.2% usefulness, demand safe)
- Stable across multicore contention (32.2% aggregate usefulness)

Key Result: Simple insertion control captures most prefetch-awareness benefits at zero cost

Future Work

Summer Vacation :

- Test with SPP (Signature Path Prefetcher) — more aggressive variant
- Run additional heterogeneous workload mixes
- Compare against DRRIP, SHiP, SRRIP policies
- Hardware cost analysis vs. PACMan and Drishti
- Conference paper submission — compile results for publication

Next Semester :

- Drishti integration: per-core prediction + prefetch-aware insertion
- Adaptive insertion position: dynamic prefetch priority with lightweight counters
- Workload characterization and final thesis documentation

Thank You